

Your Paper

You

June 25, 2026

Contents

1	Introduction	2
1.1	Certification problem	2
1.2	Incomplete verification	3
2	Preliminaries	5
2.1	Notations	5
2.2	Theoretical formulation	5
2.3	Bound propagation methods	6
2.4	Stability of neurons	7
3	Quadratic modelization of the Certification Problem	8
3.1	A targeted quadratic formulation	8
3.2	A new untargeted quadratic formulation	8
4	SDP modelization of the Certification Problem	9
4.1	From the quadratic formulation to the SDP relaxation	9
4.2	Objectives	10
4.2.1	Targeted objective	10
4.2.2	Untargeted objective	10
4.3	Constraints	10
4.3.1	Common constraints between targeted and untargeted model	10
4.3.1.1	Forward pass exact quadratic constraint	10
4.3.1.2	Bound constraint	11
4.3.1.3	Triangular constraint	11
4.3.1.4	'RLT' constraints	12
4.3.2	Specific constraints to untargeted model	12
4.3.2.1	Unique target selection constraint	12
4.3.2.2	β -binary constraint	13
4.3.2.3	Pairwise target exclusivity	13
4.3.2.4	β -consistency constraint	13
4.3.2.5	McCormick constraints on $\beta_j z_k$	14
4.3.2.6	Big-M constraints	15
4.3.2.7	More refined big-M constraints	15
4.4	Model size study	16
4.5	Generalization to other network architectures	17
4.5.1	Leaky ReLU, PReLU	17
4.5.2	Implementing pooling layers	18
4.5.2.1	Average Pooling	18

4.5.2.2	Max Pooling	18
4.5.3	Convolutional layers	18
5	Speeding up the SDP	19
5.1	Chordal decomposition	19
5.1.1	Theoretical foundation	19
5.1.2	A first chordal decomposition for the certification problem	19
5.1.3	A more general framework for chordal decomposition	20
5.2	Pruning of stable neurons	21
5.2.1	Empirical analysis of neuron stability	21
5.2.2	Pruning of stable inactive neurons	21
5.2.3	Pruning of stable active neurons	21
5.2.3.1	Linear decomposition of stable active neurons	21
5.2.3.2	Conversion to other constraints	22
5.2.3.3	Proposed approach for the case of the forward pass exact quadratic constraint	22
5.2.3.3.1	McCormick framing on non-consecutive layers outputs	23
5.2.3.3.2	Transposition to the quadratic ReLU constraint	23
5.2.3.3.3	Better choices of framings for a tighter relaxation	24
5.2.3.3.4	Selective removal of stable active neurons	25
5.2.3.3.5	Combined chordal decomposition strategy	25
5.2.4	Generalization to other activation functions	25
5.2.4.1	Leaky ReLU, PReLU	25
5.2.4.2	Smoother activations	27
5.3	Removing whole layers	28
5.3.1	Study of the matrices variable sizes	28
5.3.2	Removing the input from the variables	28
5.3.2.1	Selective pixel retention	28
5.4	Conic bundle	29
5.4.1	Static conic bundle method	29
5.4.2	Dynamic conic bundle method	30
6	Conclusion	31

Abstract

Your abstract.

1 Introduction

1.1 Certification problem

Deep Neural Networks (DNNs) have achieved remarkable success and are widely implemented in various domains, including computer vision and natural language processing. The rapid adoption of DNNs has often prioritized efficiency and automation, sometimes at the expense of safety considerations.

The research community has extensively studied various aspects of robustness, including out-of-distribution generalization, robustness to data corruption, and resistance to adversarial attacks. In particular, DNN have especially been proven vulnerable to adversarial attacks (Goodfellow, Shlens, and Szegedy 2015), where malicious actors exploit

the complexity of these models to generate examples that deceive classifiers. This issue has raised concerns in many critical domains of applications of neural networks, like autonomous vehicles or robotics, where adversarial attacks could be a means for malicious attacks.

An *adversarial attack* consists in solving a constrained optimization problem to find an adversarial example for a given input x , *i.e.*, a data in the neighborhood of x which is classified differently by the DNN. These attacks represent a significant threat, particularly when the attacker has knowledge of the model architecture and parameters. In response, two main approaches have emerged to enhance the robustness of DNNs against such attacks: adversarial training and certified defenses. Adversarial training methods aim to improve robustness by performing adversarial augmentations. While these methods do offer increased resilience, they are not foolproof and can still be vulnerable to sophisticated attacks. On the other hand, certified defenses provide mathematical guarantees of robustness against adversarial attacks.

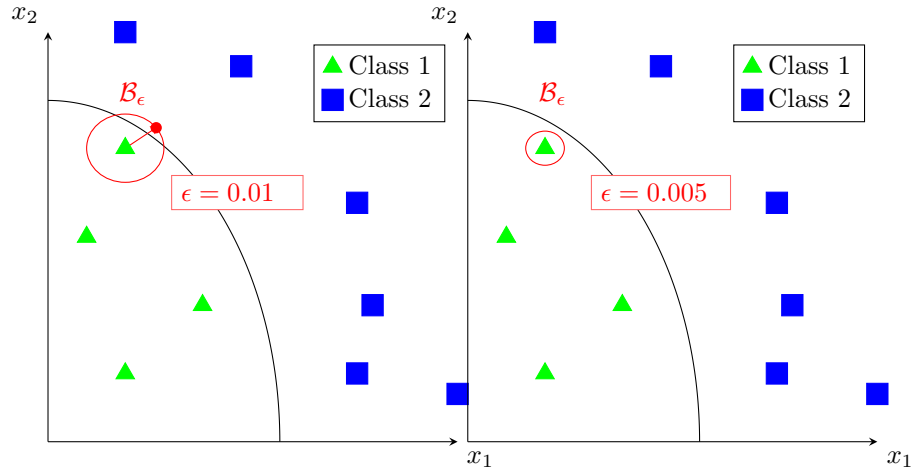


Figure 1: Illustration of the certification problem with different values of ϵ on a two-dimensional datasets divided into two classes and a L_2 -norm ball.

The certification problem for neural networks with ReLU activation functions is NP-complete (Katz, Barrett, et al. 2017). This inherent complexity implies that providing a *complete certification* requires substantial computational effort and remains limited in scalability. Many approaches solve combinatorial models to assess the stability of DNN predictions around each data. Several Mixed-Integer Programming (MIP) formulations were introduced to provide formal proofs of small ReLU DNNs (Tjeng, Xiao, and Tedrake 2019; Fischetti and Jo 2018; Cheng, Nührenberg, and Ruess 2017) but remain intractable for medium to large-scale problems.

1.2 Incomplete verification

Scalable methods often use Branch-and-Bound algorithms that rely on fast relaxations (Bunel et al. 2020; Ferrari et al. 2022; Jaeckle, Lu, and Kumar 2021; Lu and Kumar 2019), and smart branching strategies (such as neuron-level ReLU splitting (Henriksen and Lomuscio 2021)). The certification is then achieved by the solution of MIP when a given depth of the tree is reached (H. Zhang, Wang, et al. 2022). Specific algorithms were then introduced to efficiently solve these MIP formulations, such as the CROWN (H. Zhang,

Weng, et al. 2018) method and its improvements α -CROWN (Xu, H. Zhang, et al. 2021), $\alpha - \beta$ -CROWN (Wang et al. 2021), that rely on bound propagation as a pre-processing step of the Branch-and-Bound algorithm. Indeed, they compute a tight linear relaxation with a parallelized backward propagation algorithm resulting in fast computation of the root node. Then, since the resulting lower bound may be sufficient for full certification, this method combined with Branch-and-Bound proves complete verification. Other approaches rely on Satisfiability Modulo Theories (see ReLUpex (Katz, Barrett, et al. 2017)) or Marabou (Katz, D. A. Huang, et al. 2019) (Wu et al. 2024)), and aim to extend the simplex algorithm to the piecewise linear relaxation of the ReLU.

Many approaches solve combinatorial models to assess the DNNs predictions stability around each data. These approaches employ combinatorial techniques to prove that neural networks are consistent with a specification (mostly being in a ball around the data) for all possible inputs of the network. A key advantage of these models is that computing a non-negative lower bound is sufficient to certify that no adversarial attack is possible for the target class. Thus, in order to speed up the certification, many approaches solve tight semidefinite relaxations of the non-convex initial formulations to provide lower bounds. However, the current approaches are targeted ones, i.e. each combinatorial model tests if there exists an adversarial attack for one data and one *target* class. Thus, formally certifying a single data point requires looping over all possible target classes. This requirement can be rapidly cumbersome as modern datasets such as ImageNet-1k or ImageNet-21k propose hundreds or even thousands of classes. Standard non convex combinatorial models that certify DNNs are targeted ones, i.e. the model works for one data and is parameterized by a given class, implying that the formal certification of one data needs to loop over all possible classes. Since solving such models is NP-Hard, this certification approach fails for real-world datasets.

Incomplete verifiers aim to compute tight lower bounds on neural network robustness, providing formal certification guarantees whenever these bounds are sufficiently strong. A positive bound confirms robustness, while a negative one remains inconclusive, making these methods an interesting compromise between scalability and theoretical soundness. Incomplete verifiers are divided into a wide variety of approaches including convex relaxations via duality (Wong and Kolter 2018; Gwal, Dvijotham, Stanforth, Mann, et al. 2019), linear bounding of *ReLU* activations (Weng et al. 2018), or discretized input space exploration (X. Huang et al. 2017). However, most incomplete verifiers may yield conservative bounds, and even well-optimized linear relaxations can fail to produce tight lower bounds over the objective.

Semidefinite Programming (SDP) relaxations have emerged as a promising class of incomplete verifiers for neural network verification as they produce sharper lower bounds than traditional linear programming approaches. While SDP methods are computationally more intensive, foundational works (Raghunathan, Steinhardt, and Liang 2018; Richard Y Zhang 2020; Dathathri et al. 2018; Chiu and Richard Y. Zhang 2023; Lan, Zheng, and Lomuscio 2023) have demonstrated empirical tightness compared to LP relaxations. This was further advanced by integrating geometric constraints such as triangle relaxations (Batten et al. 2021) and Reformulation-Linearization Technique (RLT) cuts (Lan, Zheng, and Lomuscio 2022), which refine the feasible region for *ReLU* - activated networks. However, the relaxation becomes increasingly loose as the network depth grows, and solving SDPs for deep networks often results in scalability challenges. This phenomenon is exacerbated when dealing with multiple classes as one SDP relaxation needs to be computed *for each target class* to achieve certification.

2 Preliminaries

2.1 Notations

Notation	Space	Description
x	$\mathbb{R}^{n_0}$	Input
y	$\{1, \dots, n_K\}$	True class of x
y^*	$\{1, \dots, n_K\}$	Predicted class of x
$\hat{y}$	$\{1, \dots, n_K\}$	Selected adversarial target for x
f	$\mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_K}$	Neural network
f_k	$\mathbb{R}^{n_{k-1}} \rightarrow \mathbb{R}^{n_k}$	Layer k of the neural network
ϵ	$\mathbb{R}_+$	Radius of the adversarial ball
$\mathcal{B}_\epsilon(x)$	$\subset \mathbb{R}^{n_0}$	Adversarial ball
K	$\mathbb{N}^*$	Number of layers
n_k	$\mathbb{N}^*$	Number of neurons in layer k
$\mathcal{N}_k$	$\mathcal{P}(\{1, \dots, n_k\})$	Set of neurons in layer k
W_k	$\mathbb{R}^{n_k \times n_{k-1}}$	Weight matrix of layer k
b_k	$\mathbb{R}^{n_k}$	Bias vector of layer k
L_k, U_k	$\mathbb{R}^{n_k}$	Pre-activation lower and upper bounds at layer k within the input ball
x_k	$\mathbb{R}^{n_k}$	Post-activation value at layer k with x as input

Table 1: Summary of notation used throughout the paper.

The last layer, or logits layer, is linear : $f_K(z_{K-1}) = W_K z_{K-1} + b_K$. We suppose here that the input is well-predicted : $y = y^* = \operatorname{argmax}_{j \in \mathcal{N}_K} f_j(x)$. We suppose the bounds L_k, U_k already known for all $k \in [0, K-1]$. They are often computed in practice with bound propagation methods (see appendix).

2.2 Theoretical formulation

A general targeted theoretical formulation of the certification problem can be written as :

$$(\mathbf{Cert}_1^{\hat{y}}) \begin{cases} \min_{z_0 \in \mathbb{R}^{n_0}} & f^y(z_0) - f^{\hat{y}}(z_0) \\ \text{s.t.} & z_0 \in \mathcal{B}_\epsilon(x) \end{cases} \quad (1a)$$

or equivalently, adding the variables z_k denoting the post-activation value at layer k with z_0 as input, as:

$$(\text{Cert}_2^{\hat{y}}) \left\{ \begin{array}{ll} \min_{z \in \mathbb{R}^{\sum_{k=0}^{K-1} n_k}} & (W_K^y z_K + b_K^y) - (W_K^{\hat{y}} z_K + b_K^{\hat{y}}) \\ \text{s.t.} & z_{k+1} = f_k(z_k), \quad \forall k \in \{0, \dots, K-2\} \\ & z_0 \in \mathcal{B}_\epsilon(x) \end{array} \right. \quad \begin{array}{l} (2a) \\ (2b) \end{array}$$

When the optimal value of $(\text{Cert}_1^{\hat{y}})$ or $(\text{Cert}_2^{\hat{y}})$ is positive, the model is certified robust against the adversarial target $\hat{y}$. When the optimal value of $(\text{Cert}_1^{\hat{y}})$ or $(\text{Cert}_2^{\hat{y}})$ is negative, the model is not robust against the adversarial target $\hat{y}$. Solving the certification problem for all adversarial targets $\hat{y} \neq y$ is necessary to conclude that the model is robust.

Proposition 1. *The model is robust against **all** adversarial targets $\hat{y} \neq y$ if and only if the optimal value of $(\text{Cert}_1^{\hat{y}})$ or $(\text{Cert}_2^{\hat{y}})$ is positive for all $\hat{y} \neq y$. In other words, the model is robust if and only if $\min_{\hat{y} \neq y} \text{val}(\text{Cert}_1^{\hat{y}}) > 0$ or equivalently $\min_{\hat{y} \neq y} \text{val}(\text{Cert}_2^{\hat{y}}) > 0$.*

2.3 Bound propagation methods

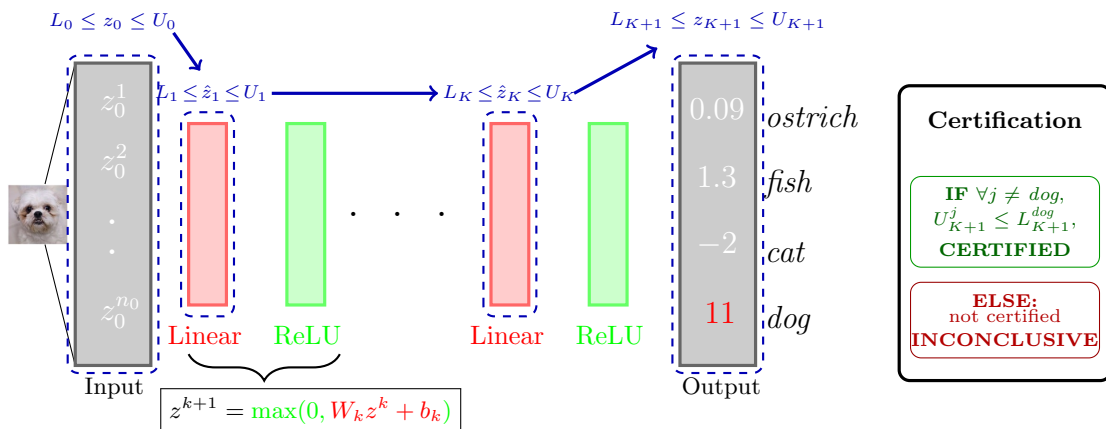


Figure 2: Bound propagation methods to certify the robustness on a single input.

Verifying the robustness of neural networks against adversarial perturbations has motivated a rich line of work based on bound propagation, which seeks to compute certified guarantees by propagating lower and upper bounds on neuron activations through the network. Early approaches relied on interval arithmetic, as formalized in Interval Bound Propagation (IBP) ((Gowal, Dvijotham, Stanforth, Bunel, et al. 2019); (Mirman, Gehr, and Vechev 2018)), which offers high scalability at the cost of loose bounds due to dependency accumulation. To tighten these estimates, linear relaxation-based methods were introduced: CROWN (H. Zhang, Weng, et al. 2018) and its GPU-accelerated variant α -CROWN (Xu, H. Zhang, et al. 2021) propagate linear bounds through non-linear activations by computing activation-specific linear approximations, yielding significantly sharper certificates. The LiRPA framework (Xu, Shi, et al. 2020) further generalizes this approach to arbitrary computational graphs. Convex relaxations of the ReLU nonlinearity, notably through the triangle relaxation, were analyzed in depth by (Salman et al.

n.d.) and form the backbone of many subsequent methods, including DeepPoly (Singh et al. 2019), which operates over a domain of affine constraints and achieves strong precision on moderately sized networks. All these methods are fundamentally incomplete: they produce certified lower bounds on the robustness margin but cannot, in general, disprove robustness, leaving a gap between the certified bound and the true worst-case margin.

2.4 Stability of neurons

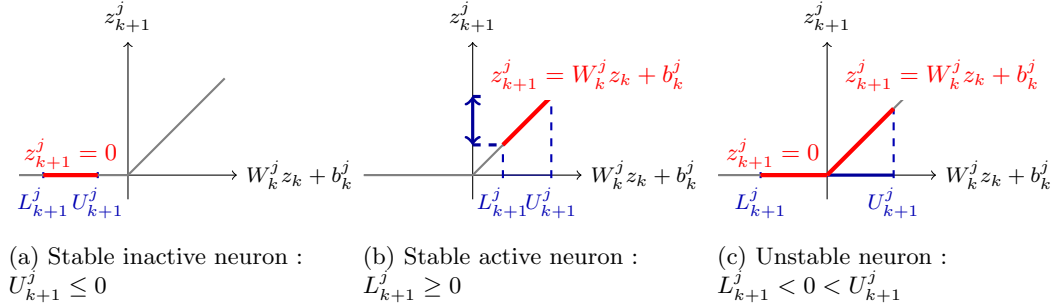


Figure 3: Illustration of the different types of stability for neuron j on layer k based on its pre-activation bounds.

Given the pre-activation bounds $[L_k^j, U_k^j]$ computed by a bound propagation method over the perturbation ball $\mathcal{B}_\epsilon(x)$, each neuron j in layer k can be classified according to its behavior across all admissible inputs. A neuron j is called *stable inactive* if $U_k^j \leq 0$: its pre-activation is always non-positive, so its ReLU output is identically zero regardless of the input perturbation. Conversely, it is *stable active* if $L_k^j \geq 0$: its pre-activation is always positive, so the ReLU acts as the identity and $z_k^j = W_{k-1}^j z_{k-1} + b_{k-1}^j$ throughout $\mathcal{B}_\epsilon(x)$. When $L_k^j < 0 < U_k^j$, the neuron is *unstable*: the ReLU may or may not activate depending on the perturbation, making this neuron the sole source of non-convexity in the certification problem. These three regimes are illustrated in Figure ?? and their associated notation is summarized in Table 2.

Notation	Space	Description
$\mathcal{N}_k^a$	$\mathcal{P}(\{1, \dots, n_k\})$	Set of stable active neurons in layer k
$\mathcal{N}_k^i$	$\mathcal{P}(\{1, \dots, n_k\})$	Set of stable inactive neurons in layer k
$\mathcal{N}_k^u$	$\mathcal{P}(\{1, \dots, n_k\})$	Set of unstable neurons in layer k
n_k^a	$\mathbb{N}^*$	Number of stable active neurons in layer k
n_k^i	$\mathbb{N}^*$	Number of stable inactive neurons in layer k
n_k^u	$\mathbb{N}^*$	Number of unstable neurons in layer k

Table 2: Notations for the activation stability of neurons.

Note that $\mathcal{N}_k = \mathcal{N}_k^a \cup \mathcal{N}_k^i \cup \mathcal{N}_k^u$ and $n_k = n_k^a + n_k^i + n_k^u$.

3 Quadratic modelization of the Certification Problem

3.1 A targeted quadratic formulation

Our work is mostly focused on Deep ReLU networks, where each hidden layer applies a ReLU activation function. Since each hidden layer applies a ReLU activation, the equality $z_{k+1} = f_k(z_k) = \max(W_k z_k + b_k, 0)$ can be equivalently expressed as the following quadratic complementarity system: $z_{k+1} \geq 0$, $z_{k+1} \geq W_k z_k + b_k$, and $z_{k+1} \odot (z_{k+1} - W_k z_k - b_k) = 0$, where $\odot$ denotes the element-wise product. The last condition enforces that for each neuron, exactly one of the two ReLU branches is active.

This reformulation yields a quadratic program (QP_T) that is the starting point for the SDP relaxation developed in Section 2.

$$(\text{QP}_T^{\hat{y}}) \left\{ \begin{array}{ll} \min & (W_K^y z_{K-1} + b_K^y) - (W_K^{\hat{y}} z_{K-1} + b_K^{\hat{y}}) \\ \text{s.t.} & z_{k+1} \geq 0, \quad z_{k+1} \geq W_k z_k + b_k, \quad \forall k \in [0, K-2] \quad (3a) \\ & z_{k+1} \odot (z_{k+1} - W_k z_k - b_k) = 0, \quad \forall k \in [0, K-2] \quad (3b) \\ & z_k \odot z_k - (l_k + u_k) \odot z_k + u_k \odot l_k \leq 0, \quad \forall k \in [0, K-1] \quad (3c) \end{array} \right.$$

The bounding constraints involve the lower and upper bounds $L_k \leq U_k$ on the pre-activation values. Note that for the case of the input layer, we have $L_0 = x - \epsilon$ and $U_0 = x + \epsilon$ (the constraint is exactly (1a) when using infinity norm). Also note that the bounding constraints enforce $z_k \in [L_k, U_k]$ (by use of second order polynomial inequalities), which is a stronger condition than the classical linear constraints.

3.2 A new untargeted quadratic formulation

We introduce in this work an untargeted formulation for the certification problem. We introduce binary variables $(\beta_j)_{j \in \mathcal{N}_K, j \neq y}$, representing the choice of the worst adversarial target. Only one β_j will be equal to 1, where the chosen class is the worst adversarial target.

The new quadratic formulation becomes :

$$(\mathbf{QP}_U) \left\{ \begin{array}{l} \min \quad (W_K^y z_{K-1} + b_K^y) - \sum_{j \in \mathcal{N}_K, j \neq y} \beta_j (W_K^j z_{K-1} + b_K^j) \\ \text{s.t.} \quad (3a) \quad (3b) \quad (3c) \\ \sum_{j \in \mathcal{N}_K, j \neq y} \beta_j = 1, \\ \beta_j \in \{0, 1\}, \quad \forall j \in \mathcal{N}_K, j \neq y. \end{array} \right. \quad (4a)$$

$$\quad \quad \quad (4b)$$

Lack of robustness against only one target class is sufficient to conclude that the model is not robust. Therefore targeting only one target class, which is the worst adversarial target, is sufficient to conclude that the model is not robust. This can be enforced by constraint (4a), which ensures that exactly one β_j is equal to 1.

Proposition 2. *The optimal value of (QP_U) is equal to the minimum of the optimal value of (QP_T) for all adversarial targets : $val(QP_U) = \min_{\hat{y} \in \mathcal{N}_K, \hat{y} \neq y} val(QP_T^{\hat{y}})$.*

Proof. 1. Let (z, β) be an optimal solution of (QP_U) . Since $\sum_{\hat{y} \neq y} \beta_{\hat{y}} = 1$ and $\beta_{\hat{y}} \in \{0, 1\}$, there exists a unique $\hat{y}^* \neq y$ such that $\beta_{\hat{y}^*} = 1$. Since z satisfies all constraints of $(QP_T^{\hat{y}^*})$, it is feasible for $(QP_T^{\hat{y}^*})$, hence:

$$val(QP_T^{\hat{y}^*}) \leq z_{\hat{y}^*}^{K+1} - z_{\hat{y}^*}^{K+1} = val(QP_U).$$

Therefore $\min_{\hat{y} \neq y} val(QP_T^{\hat{y}}) \leq val(QP_T^{\hat{y}^*}) \leq val(QP_U)$.

2. Let $\tilde{y} = \arg \min_{\hat{y} \neq y} val(QP_T^{\hat{y}})$ and let z' be an optimal solution of $(QP_T^{\tilde{y}})$. Define $\beta'_{\tilde{y}} = 1$ and $\beta'_{\hat{y}} = 0$ for all $\hat{y} \neq y, \hat{y} \neq \tilde{y}$. Then (z', β') is feasible for (QP_U) with objective value $z_{K+1}^y - z_{K+1}^{\tilde{y}} = val(QP_T^{\tilde{y}}) = \min_{\hat{y} \neq y} val(QP_T^{\hat{y}})$. Therefore $val(QP_U) \leq \min_{\hat{y} \neq y} val(QP_T^{\hat{y}})$. $\square$

4 SDP modelization of the Certification Problem

4.1 From the quadratic formulation to the SDP relaxation

The key idea behind SDP-based certification is to lift the non-convex quadratic program (2) into a higher-dimensional space where quadratic terms become linear. Let $v = (1, z_0^\top, z_1^\top, \dots, z_{K-1}^\top)^\top \in \mathbb{R}^{1+\sum_{k=0}^{K-1} n_k}$ denote the extended variable vector concatenating the bias 1 and all post-activation vectors. We define the matrix $P = vv^\top$, which is symmetric positive semidefinite (PSD) and of rank one by construction. For any scalar quantity q that is a monomial in the entries of v , we write $P[q]$ to denote the corresponding entry of P : for instance, $P[z_k^j]$ is the entry of P indexed by the component

z_k^j of v , and $P[z_k^j z_l^i]$ is the entry indexed by the product $z_k^j \cdot z_l^i$. Under this notation, any quadratic constraint on v becomes a *linear* constraint on P .

The SDP relaxation is obtained by dropping the non-convex rank-one constraint while keeping the semidefinite constraint:

$$P \succeq 0, P[1] = 1, \quad (5)$$

where $P[1]$ denotes the entry of P corresponding to the bias component. Since every rank-one PSD matrix is feasible, any solution of the original problem lifts to a feasible point of the SDP; conversely, a solution of the SDP with $\text{rank}(P) > 1$ is not in general recoverable as an exact solution. The SDP is therefore a convex relaxation of (2): its optimal value provides a certified lower bound on the robustness margin, and a non-negative value certifies robustness.

4.2 Objectives

4.2.1 Targeted objective

The targeted objective in the SDP formulation is the following :

$$\min_P (W_K^y - W_K^{\hat{y}})P[z_{K-1}] + (b_K^y - b_K^{\hat{y}})$$

4.2.2 Untargeted objective

Note that in this formulation, a new matrix variable $P = (1z\beta)^T(1z\beta)$ is introduced, which is of size $(1 + \sum_{k=0}^{K-1} n_k)^2$. The untargeted objective in the SDP formulation is the following :

$$\min_P (W_K^y P[z_{K-1}] + b_K^y) - \left(\sum_{j \in \mathcal{N}_K, j \neq y} W_K^j P[\beta_j z_{K-1}] + b_K^j P[\beta_j] \right)$$

4.3 Constraints

4.3.1 Common constraints between targeted and untargeted model

4.3.1.1 Forward pass exact quadratic constraint

Introduced in the first SDP formulation for the certification problem (Raghunathan, Steinhardt, and Liang 2018), the forward pass exact quadratic constraint can be reformulated as an SDP constraint as :

$$P[z_{k+1}] \geq 0 \quad (8a)$$

$$P[z_{k+1}] \geq W_k P[z_k] + b_k \quad (8b)$$

$$\text{diag}(P[z_{k+1}^T z_{k+1}] - W_k P[z_k^T z_k] - b_k P[z_{k+1}]) = 0 \quad (8c)$$

where (8a) and (8b) correspond to the linear inequalities of the ReLU activation (3a), and (8c) corresponds to the quadratic complementarity constraint (3b).

4.3.1.2 Bound constraint

The bound constraint (3c) can be reformulated as an SDP constraint (Raghuathan, Steinhardt, and Liang 2018) as :

$$P[(z_k^j)^2] - (L_k^j + U_k^j) \odot P[z_k^j] + U_k^j \odot L_k^j \leq 0 \quad \forall j \in \mathcal{N} \forall k \in [0, K] \quad (9a)$$

Proposition 3. *Constraint (9a) implies $L_k \leq P[z_k] \leq U_k$.*

Proof. As P is semidefinite positive, its leading principal minors are non-negative, which gives $\det \begin{bmatrix} 1 & P[z_k^j] \\ P[z_k^j] & P[(z_k^j)^2] \end{bmatrix} \geq 0$, ensuring that $P[(z_k^j)^2] \geq (P[z_k^j])^2$. Thus, (9a) implies $P[z_k^j]^2 - (L_k^j + U_k^j)P[z_k^j] + U_k^j L_k^j \leq 0$, which gives $L_k^j \leq P[z_k^j] \leq U_k^j$ by use of the second order polynome property. $\square$

4.3.1.3 Triangular constraint

The triangular constraint provides a convex upper bound on the ReLU activation function. It can be written compactly as:

$$z_{k+1} \leq A_k z_k + B_k \quad (10a)$$

where $\alpha_k = \frac{\text{ReLU}(U_k) - \text{ReLU}(L_k)}{U_k - L_k}$, $A_k = \alpha_k \odot W_k$, and $B_k = \alpha_k \odot (b_k - L_k) + \text{ReLU}(L_k)$. More precisely, depending on the activation status of neuron j of layer k , it decomposes as:

$$\begin{aligned} z_{k+1}^j &\leq 0 && \text{if } j \text{ is inactive} \\ z_{k+1}^j &\leq W_k^j z_k + b_k^j && \text{if } j \text{ is active} \\ z_{k+1}^j &\leq \frac{U_k^j}{U_k^j - L_k^j} (W_k^j z_k + b_k^j - L_k^j) && \text{if } j \text{ is unstable} \end{aligned}$$

Combined with constraints $z_{k+1}^j \geq 0$, $z_{k+1}^j \geq W_k^j z_k + b_k^j$ (3a), the triangular constraint gives $z_{k+1}^j = 0$ when neuron j is inactive, and $z_{k+1}^j = W_k^j z_k + b_k^j$ when neuron j is active. When neuron j is unstable, these constraints give 3 linear bounds plotted in Figure 4. The slope of the red line depends on the lower and upper bounds L_k^j and U_k^j of the pre-activation $W_k^j z_k + b_k^j$. The triangular constraint has been shown to be limited, as it represents only the convex hull of the output of a single ReLU neuron (Salman et al. n.d.). Recent works have proposed convex relaxations to capture the joint behavior of multiple ReLUs. Such ideas could be explored to compute more efficient cuts in SDP models. In

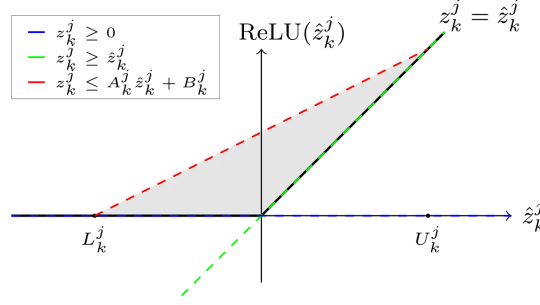


Figure 4: Triangular constraint on unstable neuron j of layer k , where $\hat{z}_k^j = W_k^j z_k + b_k^j$ is the pre-activation value. (FIGURE A REFAIRE AVEC LES BONS INDICES)

SDP models, the triangular constraint (10a), introduced in (Batten et al. 2021), can be reformulated as:

$$P[z_{k+1}^j] \leq A_k^j P[z_k] + B_k^j \quad (12a)$$

4.3.1.4 'RLT' constraints

To tighten this relaxation, (Lan, Zheng, and Lomuscio 2022) apply the Reformulation-Linearization Technique (RLT): valid linear cuts are generated by multiplying pairs of bound constraints on the pre-activation and activation vectors ($L_k \leq z_k, z_k \leq U_k$) and ReLU linear constraints ($z_{k+1} \geq 0, z_{k+1} \geq W_k z_k + b_k$, and the triangular constraint $z_{k+1} \leq A_k z_k + B_k$).

$$\text{RLT} \left\{ \begin{array}{ll} P[z_{k+1}^j z_k^l] \geq L_k^l P[z_{k+1}^j] + L_{k+1}^j P[z_k^l] - L_{k+1}^j L_k^l & \forall j \in \mathcal{N}_{k+1}, l \in \mathcal{N}_k \quad (13a) \\ P[z_{k+1}^j z_k^l] \leq U_k^l P[z_{k+1}^j] + L_{k+1}^j P[z_k^l] - L_{k+1}^j U_k^l & \forall j \in \mathcal{N}_{k+1}, l \in \mathcal{N}_k \quad (13b) \\ P[z_{k+1}^j z_k^l] \leq L_k^l P[z_{k+1}^j] + U_{k+1}^j P[z_k^l] - U_{k+1}^j L_k^l & \forall j \in \mathcal{N}_{k+1}, l \in \mathcal{N}_k \quad (13c) \\ P[(z_{k+1}^j)^2] \leq P[z_{k+1}^j] U_{k+1}^j & \forall j \in \mathcal{N}_{k+1} \quad (13d) \\ P[(z_{k+1}^j)^2] \leq A_k^j P[z_k z_{k+1}^j] + B_k^j P[z_{k+1}^j] & \forall j \in \mathcal{N}_{k+1} \quad (13e) \end{array} \right.$$

Since the full set of RLT cuts is computationally prohibitive, the authors propose an iterative algorithm that incrementally incorporates only the highest-priority cuts at each step, with priorities determined by the network weights. More precisely, for a given layer k , a given neuron $j \in \mathcal{N}_{k+1}$, and a given proportion $p \in [0, 1]$, the $\lfloor pn_k \rfloor$ cuts corresponding to the products $z_{k+1}^j z_k^l$ with the largest absolute weights $|W_{k+1}^{jl}|$ are added at each iteration. As no parameter captures *intra*-layer dependencies, cuts (13d)–(13e) are excluded from the formulation.

4.3.2 Specific constraints to untargeted model

4.3.2.1 Unique target selection constraint

Algorithm 1 Iterative RLT cut selection

Require: Neural network parameters (W, b) , pre-activation bounds (L, U) , proportion schedule $(p_r)_{r=1}^R$

Ensure: Certified lower bound

- 1: Set $r \leftarrow 1$
 - 2: Select the $\lfloor p_r n_k \rfloor$ highest-priority cuts (13a)–(13c) per layer and neuron
 - 3: Solve $\text{SDP}(p_r)$
 - 4: **while** $\text{val}(\text{SDP}(p_r)) < 0$ **and** $r < R$ **do**
 - 5: Set $r \leftarrow r + 1$
 - 6: Recompute cut priorities using the current solution and network weights
 - 7: Augment the current relaxation with the $\lfloor p_r n_k \rfloor$ highest-priority cuts
 - 8: Solve the augmented $\text{SDP}(p_r)$
 - 9: **end while**
 - 10: **return** $\text{val}(\text{SDP}(p_r))$
-

Constraint (4a) is reformulated in our SDP model as :

$$\sum_{j \in \mathcal{N}_K, j \neq y} P[\beta_j] = 1 \quad (14a)$$

4.3.2.2 β -binary constraint

As SDP relaxations do not enforce the binary nature of β_j , we add the following constraint :

$$P[(\beta_j)^2] = P[\beta_j] \quad \forall j \in \mathcal{N}_K, j \neq y \quad (15a)$$

Note that this constraint is still not sufficient to enforce the binary nature of β_j , but it provides an embedding for the variable. As P is positive semidefinite, all its leading principal minors are non-negative, which gives $P[\beta_j] \geq 0$ and $\det \begin{bmatrix} 1 & P[\beta_j] \\ P[\beta_j] & P[(\beta_j)^2] \end{bmatrix} \geq 0$, ensuring that $P[\beta_j] \leq 1$ thanks to constraint (15a).

4.3.2.3 Pairwise target exclusivity

To enforce the exclusivity of the targets, we add the following constraint :

$$P[\beta_j \beta_{j'}] = 0 \quad \forall (j, j') \in \mathcal{N}_K \times \mathcal{N}_K, j \neq y, j' \neq y, j \neq j' \quad (16a)$$

Note that this constraint is stronger than classical McCormick constraints on two binary variables. Note that the constraint : $\sum_{i \in \mathcal{N}_K, i \neq y} P[\beta_j \beta_i] = P[\beta_j]$, with proven efficiency in some problems such as QAP, is redundant in our case.

4.3.2.4 β -consistency constraint

Let $j \in \mathcal{N}_K, j \neq y$ be a target class, $k \in [1, K]$ be a layer, $l \in \mathcal{N}_k$ be a neuron in layer k . To ensure the consistency of β_j variables with the output of different layers, we can add the following cut:

$$\sum_{j \in \mathcal{N}_K, j \neq y} P[\beta_j z_k^l] = P[z_k^l] \quad \forall l \in \mathcal{N}_k, k \in [0, K] \quad (17a)$$

This cut is obtained by multiplying the constraint $\sum_{j \in \mathcal{N}_K, j \neq y} \beta_j = 1$ (4a) by z_k^l .

4.3.2.5 McCormick constraints on $\beta_j z_k$

Let $j \in \mathcal{N}_K, j \neq y$ be a target class, $k \in [0, K]$ be a layer, $l \in \mathcal{N}_k$ be a neuron in layer k . Define the effective lower bound

$$\tilde{L}_k^l = \begin{cases} L_0^l & \text{if } k = 0, \\ 0 & \text{if } k \in [1, K-1], \\ L_K^l & \text{if } k = K, \end{cases}$$

Let $T_{j,k,l}$ be the product $\beta_j z_k^l$. It can be linearized using McCormick envelopes as :

$$T_{j,k,l} \geq \tilde{L}_k^l \beta_j \quad (18a)$$

$$T_{j,k,l} \leq U_k^l \beta_j \quad (18b)$$

$$T_{j,k,l} \leq z_k^l + \tilde{L}_k^l \beta_j - \tilde{L}_k^l \quad (18c)$$

$$T_{j,k,l} \geq z_k^l + U_k^l \beta_j - U_k^l \quad (18d)$$

These constraints are valid for (QP_U).

Proposition 4. Constraints (18c) and (18d) are redundant.

Proof. 1. **Constraint (18c) is redundant.** Since $T_{i,k,l} - \tilde{L}_k^l \beta_i \geq 0$ (18a), for all $i \in \mathcal{N}_K, i \neq y, T_{j,k,l} - \tilde{L}_k^l \beta_j \leq \sum_{i \in \mathcal{N}_K, i \neq y} (T_{i,k,l} - \beta_i \tilde{L}_k^l) = z_k^l - \tilde{L}_k^l$ by constraints (17a) and (4a).,

2. **Constraint (18d) is redundant.**

$$\begin{aligned} T_{j,k,l} - z_k^l - U_k^l \beta_j + U_k^l &= T_{j,k,l} - \sum_{i \in \mathcal{N}_K, i \neq y} T_{i,k,l} - U_k^l \beta_j + U_k^l && \text{by (17a)} \\ &= - \sum_{i \in \mathcal{N}_K, i \neq y, i \neq j} T_{i,k,l} - U_k^l \beta_j + U_k^l \\ &= - \sum_{i \in \mathcal{N}_K, i \neq y, i \neq j} T_{i,k,l} - U_k^l \beta_i + \sum_{i \in \mathcal{N}_K, i \neq y} \beta_j U_k^l && \text{by (4a)} \\ &= - \sum_{i \in \mathcal{N}_K, i \neq y, i \neq j} T_{i,k,l} + \sum_{i \in \mathcal{N}_K, i \neq y, j \neq i} \beta_i U_k^l \\ &= \sum_{i \in \mathcal{N}_K, i \neq y, i \neq j} (\beta_i U_k^l - T_{i,k,l}) \\ &\geq 0 && \text{by (18b)} \end{aligned}$$

□

Proposition 5. Constraints (18a) and (18b) are not redundant.

Proof. 1. **Constraint (18a) is not redundant.** Consider two target classes: $|\mathcal{N}_K \setminus \{y\}| = 2$. Set $\beta_1 = 0, \beta_2 = 1, z_k^l = 3, U_k^l = 5, \tilde{L}_k^l = 0$, and $T_{1,k,l} = -1, T_{2,k,l} = 4$. We have $T_{1,k,l} + T_{2,k,l} = z_k^l = 3$ (beta-consistency), $T_{1,k,l} = -1 \leq U_k^l \beta_1 = 0$ (18b), and $T_{2,k,l} = 4 \leq U_k^l \beta_2 = 5$ (18b). Yet $T_{1,k,l} = -1 < 0 = \tilde{L}_k^l \beta_1$, violating (18a).

2. **Constraint (18b) is not redundant.** Consider three target classes: $|\mathcal{N}_K \setminus \{y\}| = 3$. Set $\beta_1 = \beta_2 = \beta_3 = \frac{1}{3}$, $z_k^l = 2$, $U_k^l = 3$, $\tilde{L}_k^l = 0$, and $T_{1,k,l} = \frac{3}{2}$, $T_{2,k,l} = T_{3,k,l} = \frac{1}{4}$. We have $T_{1,k,l} + T_{2,k,l} + T_{3,k,l} = z_k^l = 2$ (beta-consistency), all $T_{j,k,l} \geq 0 = \tilde{L}_k^l \beta_j$ (18a). Yet $T_{1,k,l} = \frac{3}{2} > 1 = U_k^l \beta_1$, violating (18b). $\square$

Thus, we only have to use constraints (18a) and (18b) in our model, rewritten for all $k \in [0, K-1]$ as :

$$P[\beta_j z_k^l] \geq \tilde{L}_k^l P[\beta_j] \quad (19a)$$

$$P[\beta_j z_k^l] \leq U_k^l P[\beta_j] \quad (19b)$$

These constraints are especially useful for $k = K-1$, as the product $\beta_j z_{K-1}$ appears in the objective function of (QP_U) as $-\beta_j(W_K^j z_{K-1} + b_K^j)$. The logits $z_K^j = W_K^j z_{K-1} + b_K^j$ are not directly encoded as variables in our SDP matrices. However, the McCormick bounds on $\beta_j z_K^j$ can still be expressed in terms of $P[\beta_j z_{K-1}]$, which is already present in the model. Substituting z_K^j and using $\tilde{L}_K^j = L_K^j$, constraints (18a)–(18b) at layer $k = K$ become:

$$W_K^j P[\beta_j z_{K-1}] + b_K^j P[\beta_j] \geq L_K^j P[\beta_j] \quad (20a)$$

$$W_K^j P[\beta_j z_{K-1}] + b_K^j P[\beta_j] \leq U_K^j P[\beta_j] \quad (20b)$$

4.3.2.6 Big-M constraints

The Big-M constraints exploit the pairwise structure of the target selection: if $\beta_{j_1} = 1$, then j_1 is the worst adversarial target, meaning it achieves the highest logit among all adversarial classes. For any pair $j_1 \neq j_2$ in $\mathcal{N}_K \setminus \{y\}$, this reads $z_K^{j_1} \geq z_K^{j_2}$, which we embed in a Big-M formulation:

$$z_K^{j_1} \geq z_K^{j_2} - (U_K^{j_2} - L_K^{j_1})(1 - \beta_{j_1}) \quad (21a)$$

When j_1 is not selected ($\beta_{j_1} = 0$), the constraint reduces to $z_K^{j_1} \geq z_K^{j_2} - (U_K^{j_2} - L_K^{j_1})$, which is always implied by the bound constraints $z_K^{j_1} \geq L_K^{j_1}$ and $z_K^{j_2} \leq U_K^{j_2}$, hence redundant with (9a). When j_1 is selected ($\beta_{j_1} = 1$), it enforces $z_K^{j_1} \geq z_K^{j_2}$, i.e. that j_1 achieves a higher logit than j_2 . Rewriting with our SDP variables, we have :

$$W_K^{j_1} P[z_{K-1}] + b_K^{j_1} \geq W_K^{j_2} P[z_{K-1}] + b_K^{j_2} - (U_K^{j_2} - L_K^{j_1})(1 - P[\beta_{j_1}]) \quad (22a)$$

4.3.2.7 More refined big-M constraints

While the big-M constraints (22a) enforce the ordering of the logits $z_K^{j_1}$ and $z_K^{j_2}$, the objective of (QP_U) directly involves the products $\beta_j z_K^j$. The following constraints bound these products directly, yielding tighter cuts on the relaxation:

$$\beta_{j_2} z_K^{j_2} \leq (1 - \beta_{j_1}) z_K^{j_1} + \beta_{j_2} U_K^{j_2} - (1 - \beta_{j_1}) L_K^{j_1} + \beta_{j_2} (L_K^{j_1} - z_K^{j_1}) \quad (23a)$$

$$\beta_{j_2} z_K^{j_2} \geq z_K^{j_1} - (1 - \beta_{j_2}) U_K^{j_1} \quad (23b)$$

Proposition 6. Constraints (23a) and (23b) are valid for (QP_U) .

Proof. 1. **Constraint (23a) is valid.** When $\beta_{j_1} = \beta_{j_2} = 0$, the constraint reduces to $0 \leq z_K^{j_1} - L_K^{j_1}$, which is implied by the bound constraints (3c). When $\beta_{j_1} = 0, \beta_{j_2} = 1$, it reduces to $z_K^{j_2} \leq U_K^{j_2}$, hence valid by the same bound constraints. When $\beta_{j_1} = 1, \beta_{j_2} = 0$, both sides reduce to zero.

2. **Constraint (23b) is valid.** When $\beta_{j_2} = 0$, the constraint reduces to $0 \geq z_K^{j_1} - U_K^{j_1}$, which is implied by the bound constraints (3c). When $\beta_{j_2} = 1$, it reduces to $z_K^{j_2} \geq z_K^{j_1}$, valid as j_2 is selected as the worst adversarial target. $\square$

Rewriting with our SDP variables, using $z_K^j = W_K^j P[z_{K-1}] + b_K^j$ and $\beta_j z_K^j = W_K^j P[\beta_j z_{K-1}] + b_K^j P[\beta_j]$, we obtain:

$$W_K^{j_2} P[\beta_{j_2} z_{K-1}] + b_K^{j_2} P[\beta_{j_2}] \leq (1 - P[\beta_{j_1}]) (W_K^{j_1} P[z_{K-1}] + b_K^{j_1} - L_K^{j_1}) + P[\beta_{j_2}] (U_K^{j_2} + L_K^{j_1} - W_K^{j_1} P[z_{K-1}] - b_K^{j_1}) \quad (24a)$$

$$W_K^{j_2} P[\beta_{j_2} z_{K-1}] + b_K^{j_2} P[\beta_{j_2}] \geq W_K^{j_1} P[z_{K-1}] + b_K^{j_1} - (1 - P[\beta_{j_2}]) U_K^{j_1} \quad (24b)$$

4.4 Model size study

Denoting C_m the set of layers in the chordal decomposition in the same submatrix as the β variables, the number of constraints in the SDP model is summarized in Table ??.

Constraint	Label	Theoretical Size
Baseline (Forward pass and bound)	(8c)-(9a)	$4 \sum_{k=1}^K (n_k^a + n_k^u) + n_0$
Triangular constraint	(12a)	$\sum_{k=1}^K (n_k^a + n_k^u)$
RLT cuts (all)	(13a)-(13c)	$3 \sum_{k=0}^{K-2} (n_{k+1}^a + n_{k+1}^u)(n_k^a + n_k^u)$
McCormick cuts on $\beta_j z_k$	(19a)-(19b)	$(n_K - 1) \left(\sum_{k \in C_m} (n_{k-1}^a + n_{k-1}^u) \right)$
β -consistency cuts	(17a)	$\sum_{k \in C_m} (n_{k-1}^a + n_{k-1}^u)$
Big-M cuts	(22a)	$(n_K - 1)(n_K - 2)$
Refined Big-M cuts	(24a)-(24b)	$2(n_K - 1)(n_K - 2)$

Table 3: Number of constraints in the SDP model with respect to the network size with pruning of inactive neurons

Model configuration					
Model		SDPu	SDPu	SDPu	SDPu
Network		9x100	9x100	9x200	9x200
Chordal decomposition		✓	✓	✓	✓
Pruning of inactive neurons		✓	✓	✓	✓
Pruning of active neurons		×	✓	×	✓
Constraint	Label				
Baseline (forward pass & bound)	(8c)–(9a)	3 530	3 190	4 738	4 327
Triangular constraint	(12a)	534	324	776	489
RLT cuts (all)	(13a)–(13c)	158 537	78 935	190 888	96 756
McCormick cuts on $\beta_j z_k$	(19a)–(19b)	1 510	1 510	2 267	2 267
β -consistency cuts	(17a)	84	84	126	126
Big-M cuts	(22a)	72	72	72	72
Refined Big-M cuts	(24a)–(24b)	144	144	144	144

Table 4: Mean number of constraints (over 100 data points) on different experimental settings.

4.5 Generalization to other network architectures

4.5.1 Leaky ReLU, PReLU

The PReLU and the Leaky ReLU are defined as

$$\sigma(x) = \begin{cases} \alpha x & \text{if } x \leq 0 \\ x & \text{if } x \geq 0 \end{cases} \quad (25)$$

$$(26)$$

This can be rewritten $\sigma(x) = \max(x, \alpha x)$, provided $\alpha \in (-\infty, 1)$ (which holds for the standard Leaky ReLU and contains the typical range $[0, 1)$ for a trained PReLU parameter). We propose a new quadratic formulation for the forward pass with Leaky ReLUs or PReLUs, generalizing (3a)–(3b):

$$z_{k+1} \geq \alpha(W_{k+1}z_k + b_{k+1}), \quad z_{k+1} \geq W_{k+1}z_k + b_{k+1} \quad \forall k \in [0, K-2] \quad (27)$$

$$(z_{k+1} - \alpha W_{k+1}z_k - \alpha b_{k+1}) \odot (z_{k+1} - W_{k+1}z_k - b_{k+1}) = 0 \quad \forall k \in [0, K-2] \quad (28)$$

where (28) can be rewritten $(z_{k+1}^j)^2 - (\alpha + 1)z_{k+1}^j(W_{k+1}^jz_k + b_{k+1}^j) + \alpha(W_{k+1}^jz_k + b_{k+1}^j)^2 = 0$ for each neuron j of layer $k+1$. Note that for the case $\alpha > 1$, $\sigma(x) = \min(x, \alpha x)$ and Constraint (27) are written with a reversed direction and Constraint (28) remains unchanged.

We extend our pruning technique to these activation functions in Section 5.2.4.

4.5.2 Implementing pooling layers

4.5.2.1 Average Pooling

The average pooling forward pass formulation is immediate. For a given bloc of pooling $B \subset \mathcal{N}_k$,

$$z_{k+1}^B = \frac{1}{|B|} \sum_{j \in B} z_k^j$$

4.5.2.2 Max Pooling

The maxpooling layer formulation is more complex. It is formally written as :

$$z_{k+1}^B = \max_{j \in B} z_k^j$$

For notation purposes, we order indices in $B : j_1, \dots, j_{n_B} \in B$. Bai et al. [2025](#) propose to reformulate this layer as a collection of ReLUs, and then apply the adversarial polytope computation with the triangular constraint from Wong and Kolter [2018](#). To decompose the layer into multiple ReLUs, they introduce new variables z_{k+1, j_i}^M , representing the maximum of all values before index $j_i \in B$.

$$z_{k+1, j_i}^M = \max_{j \in B, j \leq j_i} z_k^j$$

Then

$$z_{k+1}^B = z_{k+1, j_B}^M$$

Each maximum variable is created by recurrence :

$$z_{k+1, j_{i+1}}^M = z_k^{j_{i+1}} - \max(z_{k+1, j_i}^M - z_k^{j_{i+1}}, 0), \quad \forall i \in \{1, \dots, n_B\}$$

The ReLU appears with : $\max(z_{k+1, j_i}^M - z_k^{j_i}, 0) = \text{ReLU}(z_{k+1, j_i}^M - z_k^{j_i})$. The use of the triangular constraint relaxation and the dualized neural network from Wong and Kolter [2018](#) gives a bound on the output and enables certification. We propose an exact quadratic formulation to use a tighter SDP bound, still using these same variables.

$$(z_{k+1, j_{i+1}}^M - z_{k+1, j_i}^M)(z_{k+1, j_{i+1}}^M - z_k^{j_{i+1}}) = 0 \quad \forall i \in \{1, \dots, n_B\}$$

4.5.3 Convolutional layers

Convolutional layers are linear layers, with a sparse weight matrix. They can be directly used with our previous modelization.

5 Speeding up the SDP

5.1 Chordal decomposition

5.1.1 Theoretical foundation

The large SDP matrix P introduced in the previous section is costly to handle when the network is deep or wide, as its dimension grows with the total number of neurons. Chordal decomposition is a classical technique from sparse SDP theory (Vandenberghe and Andersen 2015) that exploits the sparsity of P to replace a single large semidefinite constraint by several coupled constraints on smaller submatrices. A graph $G = (V, E)$ is *chordal* if every cycle of length four or more has a chord, i.e. an edge connecting two non-adjacent cycle vertices. For a symmetric matrix A whose off-diagonal zero pattern is encoded by the complement of G (i.e. $A_{ij} = 0$ whenever $(i, j) \notin E$), the following holds:

Theorem 1 (Chordal decomposition (Agler et al. 1988)). *Let G be a chordal graph with maximal cliques $\mathcal{C}_1, \dots, \mathcal{C}_m$. A symmetric matrix A with sparsity pattern G is positive semidefinite if and only if every principal submatrix $A[\mathcal{C}_s]$ is positive semidefinite:*

$$A \succeq 0 \iff A[\mathcal{C}_s] \succeq 0 \quad \forall s \in \{1, \dots, m\}.$$

5.1.2 A first chordal decomposition for the certification problem

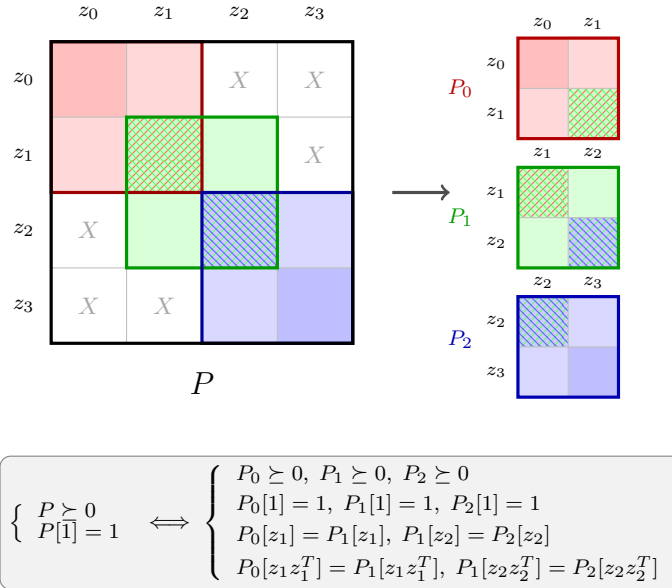


Figure 5: Chordal decomposition of the SDP matrix P into sub-matrices P_0, P_1, P_2 for a network with 4 layers.

In the certification problem, the constraints only involve products $z_k^j z_l^i$ between neurons of *consecutive* layers ($|k - l| \leq 1$). The sparsity graph of P have for nodes the components of

$v = (1, z_0, \dots, z_{K-1})$ and for edges connected pairs that appear in at least one constraint. Consequently, the sparsity graph of P has a particular structure: layer k is connected to layers $k-1$ and $k+1$, but not to any layer $k' \leq k-2$. Such a graph is chordal (a path graph has no cycle of length ≥ 4), and its maximal cliques are exactly

$$\mathcal{C}_k = \{1\} \cup \mathcal{N}_k \cup \mathcal{N}_{k+1}, \quad k = 0, \dots, K-2.$$

Applying the theorem, the constraint $P \succeq 0$ is equivalent to requiring each submatrix $P_k = P[\mathcal{C}_k]$ to be positive semidefinite. Since adjacent cliques $\mathcal{C}_k$ and $\mathcal{C}_{k+1}$ share the variables of layer $k+1$, the submatrices P_k and P_{k+1} must agree on their common entries. These *consistency constraints* take the form:

$$P_k[z_{k+1}^j] = P_{k+1}[z_{k+1}^j] \quad \forall j \in \mathcal{N}_{k+1} \quad (29a)$$

$$P_k[z_{k+1}^j z_{k+1}^i] = P_{k+1}[z_{k+1}^j z_{k+1}^i] \quad \forall (j, i) \in \mathcal{N}_{k+1}^2 \quad (29b)$$

Chordal decomposition for the certification problem has been introduced in (Batten et al. 2021) making significant speed-ups. When *all* consistency constraints (29a)–(29b) are enforced, the decomposition is exact and the optimal value is unchanged. In practice, the $\frac{n_{k+1}(n_{k+1}+3)}{2}$ quadratic constraints (29b) are costly, so we retain only the n_{k+1} linear constraints (29a), yielding a relaxation whose optimal value is a lower bound on the full SDP value.

5.1.3 A more general framework for chordal decomposition

The pair-wise decomposition described above assumes that the quadratic terms of the SDP only involve neurons from *consecutive* layers. This property is exploited in the next section to prune stable active neurons: each stable active neuron z_k^a (with $L_k^a \geq 0$) is removed from the SDP variables and replaced by its exact linear expansion in terms of the unstable neurons of all preceding layers $l < k$. After this substitution, two types of cross-layer terms arise in the constraints:

- **Linear cross-layer terms**, of the form z_l^u for $l < k-1$: these appear in the bias row and column of existing submatrices P_l , and are therefore already present in the current decomposition without any modification.
- **Quadratic cross-layer terms**, of the form $z_l^u z_{l'}^w$ for non-consecutive layers $l \neq l'$: these products are not indexed by any submatrix P_r in the pair-wise decomposition, and cannot be represented as SDP variables within it.

For the quadratic cross-layer terms, the simplest remedy is to bound each product $z_l^u z_{l'}^w$ by a pair of McCormick linear constraints using the known bounds $[0, U_l^u]$ and $[0, U_{l'}^w]$. This keeps the submatrix sizes small but introduces an additional relaxation on the cross-layer products.

A tighter alternative is to use a *generalized chordal decomposition* with larger cliques that span several consecutive layers. The corresponding submatrix captures all products between neurons belonging to the same group, including cross-layer ones. Consistency constraints between adjacent groups are imposed on the shared boundary layer, exactly as in the pair-wise case. All products between neurons of *different* groups still require McCormick bounds.

This coarser decomposition yields a tighter SDP relaxation since cross-layer quadratic products are represented exactly as semidefinite entries rather than approximated by linear bounds.

5.2 Pruning of stable neurons

5.2.1 Empirical analysis of neuron stability

5.2.2 Pruning of stable inactive neurons

For a stable inactive neuron, the pre-activation value is always non-positive, so the ReLU always clips it to zero: $z_k^j = 0$ for all feasible z_0 . This deterministic behaviour allows one to substitute $z_k^j = 0$ everywhere in the SDP formulation, which removes the corresponding row and column from the matrix, and simplifies all constraints that involve z_k^j (with one variable less). Some constraints entirely disappear (all bound and relu constraints on inactive neurons (8c) (9a)). This pruning has been introduced in (Batten et al. 2021) achieving significant speed-ups.

5.2.3 Pruning of stable active neurons

Unlike stable inactive neurons, stable active neurons cannot be directly removed from the model since their output is a non-trivial function of the network inputs. We propose a new pruning strategy in which each stable active neuron is substituted by its exact linear expansion in terms of the unstable neuron outputs of all preceding layers. This substitution eliminates the corresponding variables from the SDP matrices, reducing the problem size, at the cost of a moderate increase in constraints' number.

5.2.3.1 Linear decomposition of stable active neurons

Each stable active neuron a of layer k can be formulated as a linear combinations of previous layers unstable outputs u , *i.e.*

$$z_k^a = \sum_{l=0}^{k-1} \sum_{u=1: u \text{ unstable}}^{n_l} \lambda_l^{u(a,k)} z_l^u + \gamma(a,k)$$

where $\gamma(a,k) \in \mathbb{R}$ can be viewed as the bias of the expression of z_k^a , and $\lambda_l^{u(a,k)} \in \mathbb{R}$ is the linear coefficient of neuron u of layer l in the expression of z_k^a . We thus have to compute for each *stable active* neuron a of layer k , and for all *unstable* neuron u of layer $l \in [0, k-1]$ the coefficients $\lambda_l^{u(a,k)}$ and the bias term $\gamma(a,k)$.

For each stable active neuron a of layer k , these coefficients can then be computed dynamically as follows :

- if $k = 1$, and since all neurons u of layer 0 are *unstable*, we have for all stable active neuron a of layer 1:

$$z_1^a = \sum_{i=0}^{n_0} W_{1,i}^a z_0^i + b_1^a$$

and therefore $\lambda_0^{u(a,1)} = W_{1,u}^a$, and $\gamma(a,1) = b_1^a$;

- if $k > 1$, for a stable active neuron a on such layer,

$$\begin{aligned}
z_k^a &= \sum_{i=1: i \text{ stable or unstable}}^{n_{k-1}} W_{k,i}^a z_{k-1}^i + b_k^a \\
&= \sum_{u=1: u \text{ unstable}}^{n_{k-1}} W_{k,u}^a z_{k-1}^u + \sum_{r=1: r \text{ active}}^{n_{k-1}} W_{k,r}^a z_{k-1}^r + b_k^a \\
&= \sum_{u=1: u \text{ unstable}}^{n_{k-1}} W_{k,u}^a z_{k-1}^u \\
&\quad + \sum_{r=1: r \text{ active}}^{n_{k-1}} W_{k,r}^a \left(\sum_{l=0}^{k-2} \sum_{u=1: u \text{ unstable}}^{n_l} \lambda_l^u(r, k-1) z_l^u + \gamma(r, k-1) \right) + b_k^a \\
&= \sum_{u=1: u \text{ unstable}}^{n_{k-1}} \underbrace{W_{k,u}^a}_{\lambda_{k-1}^u(a, k)} z_{k-1}^u + \sum_{l=0}^{k-2} \sum_{u=1: u \text{ unstable}}^{n_l} \underbrace{\sum_{r=1: r \text{ active}}^{n_{k-1}} W_{k,r}^a \lambda_l^u(r, k-1)}_{\lambda_l^u(a, k)} z_l^u \\
&\quad + \underbrace{\sum_{r=1: r \text{ active}}^{n_{k-1}} W_{k,r}^a \gamma(r, k-1)}_{\gamma(a, k)} + b_k^a
\end{aligned}$$

Which gives : For each stable active neuron a of layer $k > 1$:

$$\begin{aligned}
- \gamma(a, k) &= \sum_{r=1: r \text{ active}}^{n_{k-1}} W_{k,r}^a \gamma(r, k-1) + b_k^a \\
- \text{For all unstable neuron } u \text{ on layer } k-1 : \lambda_{k-1}^u(a, k) &= W_{k,u}^a \\
- \text{For all unstable neuron } u \text{ on layer } l \leq k-2 : \\
\lambda_l^u(a, k) &= \sum_{r=1: r \text{ active}}^{n_{k-1}} W_{k,r}^a \lambda_l^u(r, k-1)
\end{aligned}$$

5.2.3.2 Conversion to other constraints

5.2.3.3 Proposed approach for the case of the forward pass exact quadratic constraint

Using the above coefficients, our *ReLU* quadratic constraint can be rewritten

$$\begin{aligned}
z_{k+1}^j(z_{k+1}^j - \sum_{u=1: u \text{ unstable}}^{n_k} (W_{k+1,u}^j z_k^u)) - b_{k+1}^j &= z_{k+1}^j (\sum_{a=1: a \text{ active}}^{n_k} W_{k+1,a}^j z_k^a) \\
&= \sum_{a=1: a \text{ active}}^{n_k} W_{k+1,a}^j z_{k+1}^j z_k^a \\
&= \sum_{a=1: a \text{ active}}^{n_k} W_{k+1,a}^j z_{k+1}^j \left(\sum_{l=0}^{k-2} \sum_{u=1: u \text{ unstable}}^{n_l} \lambda_l^u(a,k) z_l^u + \gamma(a,k) \right) \\
&= \sum_{l=0}^{k-2} \sum_{u=1: u \text{ unstable}}^{n_l} \underbrace{\sum_{a=1: a \text{ active}}^{n_k} W_{k+1,a}^j \lambda_l^u(a,k) z_{k+1}^j z_l^u}_{A_l^u(j,k+1)} \\
&\quad + \underbrace{\sum_{a=1: a \text{ active}}^{n_k} W_{k+1,a}^j \gamma(a,k) z_{k+1}^j}_{B(j,k+1)} \\
z_{k+1}^j(z_{k+1}^j - \sum_{u=1: u \text{ unstable}}^{n_k} (W_{k+1,u}^j z_k^u)) - b_{k+1}^j &= \sum_{l=0}^{k-2} \sum_{u=1: u \text{ unstable}}^{n_l} A_l^u(j,k+1) z_{k+1}^j z_l^u + B(j,k+1) z_{k+1}^j
\end{aligned}$$

Or equivalently:

$$z_{k+1}^j(z_{k+1}^j - \sum_{u=1: u \text{ unstable}}^{n_k} (W_{k+1,u}^j z_k^u)) - b_{k+1}^j - B(j,k+1) z_{k+1}^j = \sum_{l=0}^{k-2} \sum_{u=1: u \text{ unstable}}^{n_l} A_l^u(j,k+1) z_{k+1}^j z_l^u \quad (30)$$

where for each unstable neuron j of layer $k+1$, we have:

- $A_l^u(j,k+1) = \sum_{a=1 \text{ active}}^{n_k} W_{k+1,a}^j \lambda_l^u(a,k)$ for all $u \text{ unstable} \in \{1, \dots, n_l\}, l \leq k-1$
- $B(j,k+1) = \sum_{a=1 \text{ active}}^{n_k} W_{k+1,a}^j \gamma(a,k)$

5.2.3.3.1 McCormick framing on non-consecutive layers outputs

We now consider the quadratic terms $z_{k+1}^j z_l^u$ involved in Constraints (30). Observe that for a neuron j of layer $k+1$, the variables z_l^u stands for a layer l that precedes layer k by a distance of at least 2, *i.e.*, $l \in [k-1]$, and u is an unstable neuron of the associated layer l . Since we use chordal decomposition in our SDP relaxation, only products of neurons of two consecutive layers are modeled and the quadratic terms $z_{k+1}^j z_l^u$ are not considered in our formulation. Consequently, we cannot directly add Constraint (30) in our SDP. To keep a *ReLU* constraint in our formulation, our idea is therefore to upper and lower bound the left-hand side of Constraint (30) using the McCormick envelopes (McCormick 1976) of the products $z_{k+1}^j z_l^u$.

We have $0 \leq z_{k+1}^j \leq U_{k+1}^j$, $L_l^u \leq z_l^u \leq U_l^u$. Note that we compute U_{k+1}^j , U_l^u and L_l^u with β -CROWN, and for $l \geq 1$, we take $L_l^u = 0$ as it is a better bound than the lower bound on the preactivation vector. The McCormick envelopes stand for:

$$z_{k+1}^j z_l^u \leq \min\{U_{k+1}^j z_l^u + L_l^u z_{k+1}^j - U_{k+1}^j L_l^u, U_l^u z_{k+1}^j\} \quad (31)$$

$$z_{k+1}^j z_l^u \geq \max\{L_l^u z_{k+1}^j, U_l^u z_{k+1}^j + U_{k+1}^j z_l^u - U_{k+1}^j U_l^u\} \quad (32)$$

5.2.3.3.2 Transposition to the quadratic ReLU constraint

To upper bound Constraints (30), we generate the 2 following expressions:

$$\left\{ \begin{array}{l} UP_1 = \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) > 0}}^{n_l} A_l^u(j,k+1)(U_{k+1}^j z_l^u + L_l^u z_{k+1}^j - U_{k+1}^j L_l^u) \\ \quad - \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) < 0}}^{n_l} A_{k+1}^j(j,k+1) L_l^u z_{k+1}^j \\ UP_2 = \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) > 0}}^{n_l} A_{k+1}^j(j,k+1) U_l^u z_{k+1}^j \\ \quad - \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) < 0}}^{n_l} A_{k+1}^j(j,k+1)(U_l^u z_{k+1}^j + U_{k+1}^j z_l^u - U_{k+1}^j U_l^u) \end{array} \right. \quad (33)$$

(34)

To lower bound Constraints (30), we generate the 2 following expressions:

$$\left\{ \begin{array}{l} LO_1 = \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) > 0}}^{n_l} A_{k+1}^j(j,k+1) L_l^u z_{k+1}^j \\ \quad - \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) < 0}}^{n_l} A_l^u(j,k+1)(U_{k+1}^j z_l^u + L_l^u z_{k+1}^j - U_{k+1}^j L_l^u) \\ LO_2 = \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) > 0}}^{n_l} A_{k+1}^j(j,k+1) U_l^u z_{k+1}^j \\ \quad - \sum_{l=0}^{k-2} \sum_{\substack{u=1: \text{ } u \text{ unstable} \\ A_l^u(j,k+1) < 0}}^{n_l} A_{k+1}^j(j,k+1)((U_l^u z_{k+1}^j + U_{k+1}^j z_l^u - U_{k+1}^j U_l^u)) \end{array} \right. \quad (35)$$

(36)

To sum up, we relax Constraints (30) as follows:

$$\left\{ \begin{array}{l} z_{k+1}^j (z_{k+1}^j - \sum_{u=1: \text{ } u \text{ unstable}}^{n_k} (W_{k+1,u}^j z_k^u)) - b_{k+1}^j \\ \quad - B(j,k+1) z_{k+1}^j \leq UP_r \\ z_{k+1}^j (z_{k+1}^j - \sum_{u=1: \text{ } u \text{ unstable}}^{n_k} (W_{k+1,u}^j z_k^u)) - b_{k+1}^j \\ \quad - B(j,k+1) z_{k+1}^j \geq LO_r \end{array} \right. \quad \forall r = 1, 2 \quad (37)$$

(38)

5.2.3.3 Better choices of framings for a tighter relaxation

The framing described above bundles all cross-layer products into a single global upper bound UP_r and lower bound LO_r . However, for each individual product $z_{k+1}^j z_l^u$, the McCormick envelope provides two valid upper bounds and two valid lower bounds (equations (31)–(32)). The global expressions UP_1 , UP_2 , LO_1 , LO_2 each correspond to a particular assignment of one of the two bounds to every product simultaneously. In principle, one could choose independently for each product which bound to apply, yielding 2^M possible upper framings and 2^M lower framings, where $M = \sum_{l=0}^{k-2} |\mathcal{N}_l^{\text{uns}}|$ is the total number of cross-layer products. Selecting the tightest overall framing is therefore combinatorially hard.

In practice, a simple and effective strategy is to add *all four* constraints UP_1 , UP_2 , LO_1 , LO_2 simultaneously. The solver then automatically exploits the tightest active constraint at each point: imposing both $LHS \leq UP_1$ and $LHS \leq UP_2$ is equivalent to imposing $LHS \leq \min(UP_1, UP_2)$, and similarly for the lower bounds. This incurs only four extra constraints per unstable neuron j of layer $k + 1$, and yields the tightest relaxation achievable within the family of framings that use the same bound type for every product in the sum. Another heuristic is to randomly chose between the two bound options for each quadratic product.

5.2.3.3.4 Selective removal of stable active neurons

Pruning all stable active neurons simultaneously may generate an excessive number of McCormick framings. A stable active neuron at layer k produces cross-layer products $z_{k+1}^j z_l^u$ for all $l \leq k - 1$, so pruning deep stable actives yields products spanning the entire depth of the network, resulting in dense constraint matrices and a substantial loss of tightness.

To control this overhead, we propose to prune stable active neurons *selectively*: only neurons at layers $k \leq l_{\max}$ are substituted by their linear expansion, while stable active neurons at layers $k > l_{\max}$ are retained as explicit SDP variables. Selecting $l_{\max} \in \{1, \dots, K - 1\}$ has two complementary effects. First, it limits the reach of cross-layer products: a substituted neuron at layer $k \leq l_{\max}$ only generates products involving layers 0 through $k - 1 \leq l_{\max} - 1$, keeping the McCormick framing sparse and the bounds tight. Second, it concentrates the pruning effort on the layers where it is most beneficial: early layers tend to have the highest density of stable active neurons, and the activation bounds $[L_k^u, U_k^u]$ computed by α -CROWN are tightest for shallow layers, so the McCormick approximation is most accurate there. Concretely, the quadratic ReLU constraints (8c) are written exactly for $k > l_{\max} + 1$, and relaxed via McCormick framing for $k \leq l_{\max} + 1$. The parameter $l_{\max}$ thus provides an explicit trade-off between the size reduction gained from pruning and the tightness of the resulting SDP relaxation.

5.2.3.3.5 Combined chordal decomposition strategy

All in all, even if the boundings with McCormick framing are tight, the great number of quadratic product to embed with these framings may lead to a final relaxed constraint that can be too loose (in addition to being very dense and not sparse, which is very difficult for the solver to solve). Adapting the chordal decomposition strategy 5.1.3 to the new constraints is thus crucial to keep a good balance between the tightness of the relaxation and the computational time.

5.2.4 Generalization to other activation functions

5.2.4.1 Leaky ReLU, PReLU

This stable active pruning method applies to any activation function σ , which coincides with the identity map above some threshold: $\sigma(x) = x$, $\forall x \in [T, \infty)$ or $\sigma(x) = x$, $\forall x \in (-\infty, T]$. Whenever the lower bound (resp. upper bound) of a neuron j of layer k satisfies $L \geq T$ (resp. $U \leq T$), the neuron lies entirely in its linear regime and can therefore be replaced by a linear decomposition on previous layer's outputs.

More generally, this decomposition is available for any activation function that coincides with a linear map beyond some threshold $\sigma(x) = \alpha x + \Delta$, $\forall x \in [T, \infty)$ or

$\sigma(x) = \alpha x + \Delta$, $\forall x \in (-\infty, T]$. This is the case of the PReLU and the Leaky ReLU, where $\sigma(x) = \alpha x$ if $x \leq 0$. For these two functions, all neurons satisfying $U \leq 0$ or $L \geq 0$ can be immediately pruned and replaced. Note that for these activation functions, our linear-decomposition method becomes the *only* available pruning mechanism, since no neuron can be pruned for free under the 'stable inactive' paradigm (which only applies to ReLU, where the inactive branch collapses to zero). The expected number of prunable neurons is therefore significantly larger than in the ReLU case.

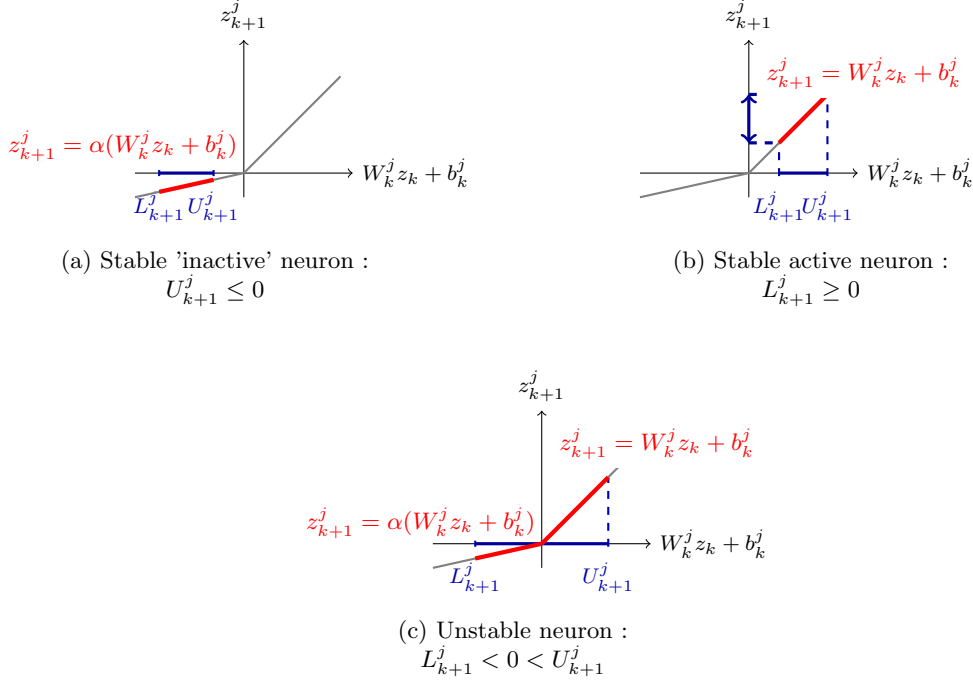


Figure 6: Illustration of the different types of stability modes for the activation function PReLU or Leaky ReLU for neuron j on layer k based on its pre-activation bounds.

Linear decomposition. Let a be a stable (active **or** inactive) neuron on layer k . Then $z_k^a = W_k^a z_{k-1} + b_k^a$ or $z_k^a = \alpha W_k^a z_{k-1} + \alpha b_k^a$. The same dynamic algorithm than in Section 5.2.3.1, yields a linear decomposition of this neuron into all previous layers's remaining neurons : $z_k^a = \sum_{l=0}^{k-1} \sum_{u=1: u \text{ unstable}}^{n_l} \lambda_l^u(a, k) z_l^u + \gamma(a, k)$.

Forward pass quadratic constraint. As presented in Section 4.5.1, the forward pass in the neural network can be rewritten from (3a)-(3b) to a more general form:

$$z_{k+1} \geq \alpha(W_k z_k + b_k), \quad z_{k+1} \geq W_k z_k + b_k \quad \forall k \in [0, K-2] \quad (27)$$

$$(z_{k+1}^j)^2 - (\alpha + 1)z_{k+1}^j(W_{k+1}^j z_k + b_{k+1}^j) + \alpha(W_{k+1}^j z_k + b_{k+1}^j)^2 = 0 \quad \forall k \in [0, K-2], \quad \forall j \in \mathcal{N}_k \quad (28)$$

The forward pass quadratic can be rewritten :

$$\begin{aligned} & (z_{k+1}^j)^2 - (\alpha + 1)z_{k+1}^j(W_{k+1,u}^j z_k^u + b_k) + \alpha(W_{k+1,u}^j z_k^u + b_k)^2 + 2\alpha b_k W_{k+1,a}^j z_k^a \\ & = (\alpha + 1)z_{k+1}^j(W_{k+1,a}^j z_k^a) - 2\alpha W_{k+1,u}^j z_k^u W_{k+1,a}^j z_k^a - \alpha(W_{k+1,a}^j z_k^a)^2 \end{aligned}$$

- Left part. The terms $(z_{k+1}^j)^2$ and $-(\alpha+1)z_{k+1}^j(W_k z_k + b_k)$ remain, up to the linear factor $\alpha+1$, the same terms as in our pruning method for ReLU neural networks. $\alpha(W_{k+1,u}^j z_k^u + b_k)^2$ involve linear terms on variables of layer k and quadratic products intra-layer of same variables. They can be directly added into the constraint. $2\alpha b_k W_{k+1,a}^j z_k^a$ can be rewritten with the linear decomposition of stable active neurons, and will give linear terms on unstable neurons on previous layers $l \leq k-1$. They can be directly added in the constraint.
- Right part. $(\alpha+1)z_{k+1}^j(W_{k+1,a}^j z_k^a)$ remains, up to the linear factor $\alpha+1$, the same term as in our pruning method, and can be bounded with McCormick envelopes. Substituting the stable active neurons by their linear equivalence in $-2\alpha W_{k+1,u}^j z_k^u W_{k+1,a}^j z_k^a$ gives quadratic products for consecutive layers k and $k-1$, which can be directly added, and quadratic products on non-consecutive layers $l < k-1$ and $k-1$, which need McCormick envelopes. Applying the same decomposition to $\alpha(W_{k+1,a}^j z_k^a)^2$ gives quadratic terms on consecutive layers $0 \leq l, l' \leq k-1, |l-l'| \leq 1$ (with direct inclusion of variables) and non-consecutive ones $0 \leq l, l' \leq k-1, |l-l'| > 1$ needing McCormick bounds.

The term $\alpha(W_k z_k + b_k)^2$ is new compared to the ReLU case. It involves linear terms, as well as quadratic products between both consecutive and non-consecutive layers.

If $\alpha = 0$, the formulation reduces to that of the ReLU pruning method.

5.2.4.2 Smoother activations

The pruning method described above relies on the existence of an exact linear regime, which does not hold for smoother activations such as GeLU or Swish: these functions never exactly coincide with a linear map on an interval, so no neuron can be losslessly substituted by a linear expansion. A relaxed variant of the approach may nonetheless be applicable. Instead of an exact substitution, one could replace σ by a linear *lower bound* $\sigma(x) \geq (1-\delta)x$ valid on the neuron's pre-activation range $[L, U]$, with δ chosen as a function of L (and possibly U) so that the bound is tight where the neuron is most likely to operate.

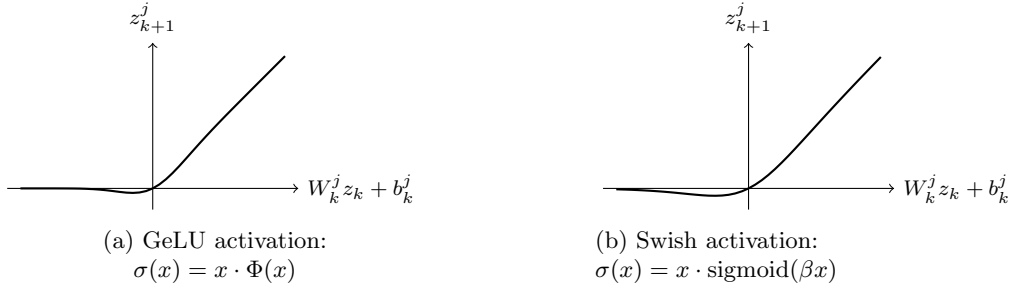


Figure 7: The true activation functions GeLU and Swish, plotted on the pre-activation range $[-5, 5]$.

Note that such a substitution would turn the pruning step into an additional source of relaxation, in the same way that the existing bound and triangularization constraints already relax the exact ReLU behaviour, with δ controlling the trade-off between the size reduction and the tightness of the resulting SDP. Investigating this trade-off, and how it composes with the McCormick framing of cross-layer products, is left for future work.

5.3 Removing whole layers

5.3.1 Study of the matrices variable sizes

5.3.2 Removing the input from the variables

For most image classification networks, the input dimension n_0 is an order of magnitude larger than the hidden layer widths (e.g. 784 versus 100 for a MNIST network). Since the submatrix $P_0 = P[\{1, z_0, z_1\}]$ has dimension $1 + n_0 + n_1$, retaining the input z_0 as an SDP variable dominates the problem size. We therefore consider an additional reduction in which z_0 is removed entirely from the set of SDP variables.

When z_0 is no longer a variable, the adversarial ball $\mathcal{B}_\epsilon(x)$ is no longer encoded explicitly in the SDP. Its effect is instead absorbed implicitly into the pre-activation bounds $[L_1^j, U_1^j]$ computed by α -CROWN, which already account for all admissible inputs. This substitution is a valid relaxation: the resulting SDP remains a lower bound on the true robustness margin, but the bound may be weaker since the coupling between the input perturbation and the first hidden layer is no longer represented.

Removing z_0 affects the constraints involving the first layer. The quadratic ReLU constraint (8c), the linear ReLU constraint (8b), and the triangular constraint (12a) for $k = 1$ all explicitly involve z_0 and must be dropped. In contrast, the bound constraints on z_1 (9a) and the beta-consistency and McCormick constraints (17a)–(19b) do not involve z_0 and remain valid. Additionally, stable active neurons at layer $k = 1$ cannot be pruned under this setting, since their linear decomposition $z_1^a = \sum_i W_{1,i}^a z_0^i + b_1^a$ contains z_0 terms that are no longer SDP variables.

5.3.2.1 Selective pixel retention

Removing all input variables may over-relax the SDP. A natural compromise is to retain a fraction p of the input coordinates as variables, namely those whose contribution to the first layer is largest. Denoting $\mathcal{N}_0^{\text{var}} \subseteq \mathcal{N}_0$ the set of retained input coordinates, and using the pre-activation bounds L_0^i, U_0^i to substitute the non-retained inputs by their worst-case values, the linear ReLU and triangular constraints for $k = 1$ are adapted as:

$$z_1^j \geq \sum_{i \in \mathcal{N}_0^{\text{var}}} W_1^{j,i} z_0^i + \sum_{i \notin \mathcal{N}_0^{\text{var}}} W_1^{j,i} L_0^i + b_1^j, \quad \forall j \in \mathcal{N}_1, \quad (39)$$

$$z_1^j \leq \sum_{i \in \mathcal{N}_0^{\text{var}}} A_1^{j,i} z_0^i + \sum_{i \notin \mathcal{N}_0^{\text{var}}} A_1^{j,i} U_0^i + B_1^j, \quad \forall j \in \mathcal{N}_1. \quad (40)$$

The selection of $\mathcal{N}_0^{\text{var}}$ is guided by the importance of each input coordinate in the first layer transformation. For an L_∞ adversarial ball, a natural criterion is to rank coordinates i by the L_1 norm $\sum_j |W_1^{j,i}|$ of the corresponding column of W_1 , and retain the top- p variables. For an L_2 ball, the L_2 column norm is the appropriate criterion as the complementary norm. The parameter $p \in [0, 1]$ continuously interpolates between the fully reduced model ($p = 0$, all input variables removed) and the full model ($p = 1$, all input variables retained).

5.4 Conic bundle

5.4.1 Static conic bundle method

Reformulating the SDP certification problem as :

$$\mathbf{SDP}_{CB} \begin{cases} \max \langle Q, P \rangle \\ \text{s.t. } \langle Q_r, P \rangle - d_r \leq 0 \\ P \succeq 0 \end{cases} \quad r \in \mathcal{R}$$

$\langle \cdot, \cdot \rangle$ denotes the Frobenius inner product. Note that each constraint, whether it is an inequality or an equality, can be reformulated as an inequality constraint by introducing a slack variable in P . Without loss of generality, we only consider the case where the constraints in $\mathcal{R}_D$ are inequalities. Thus we consider that $\mathcal{R}$ is the set of all constraints (consisting of a combination of the constraints described in Section 4.3), P is the matrix variable of the SDP (with potentially additional slack variables), Q is the matrix variable corresponding to the objective, and Q_r and d_r are the matrices and scalars corresponding to the constraints.

The principal idea of the conic bundle method is to dualize some of the constraints to reduce the size of the primal SDP. Introducing $\mathcal{R}_D \subseteq \mathcal{R}$ the set of constraints to dualize. We write $g_r(P) = \langle Q_r, P \rangle - d_r$ for all $r \in \mathcal{R}_D$.

We note θ_r the dual variable associated to the constraint $g_r(P) \leq 0$. The partial Lagrangian of the problem is then defined as :

$$\mathcal{L}(P, \theta) = \langle Q, P \rangle + \sum_{r \in \mathcal{R}_D} \theta_r g_r(P)$$

The dual function is then defined as :

$$\begin{aligned} h(\theta) &= \max_{\substack{g_r(P) \leq 0, \forall r \in \mathcal{R} \setminus \mathcal{R}_D \\ P \succeq 0}} \mathcal{L}(P, \theta) \\ &= \max \quad \langle Q + \sum_{r \in \mathcal{R}_D} \theta_r Q_r, P \rangle - \sum_{r \in \mathcal{R}_D} \theta_r d_r \\ \text{s.t.} \quad &\langle Q_r, P \rangle - d_r \leq 0, \quad \forall r \in \mathcal{R} \setminus \mathcal{R}_D \\ &P \succeq 0 \end{aligned}$$

The dual of $(\mathbf{SDP}_{CB})$ is then defined as :

$$\mathbf{SDP}_{CB}^D : \min_{\theta \geq 0} h(\theta)$$

As $\text{val}(\mathbf{SDP}_{CB}) = \text{val}(\mathbf{SDP}_{CB}^D)$, we can solve the original problem by solving the dual problem.

Proposition 7. *A sub-gradient of h at a given θ is $(g_r(P^*(\theta)))_{r \in \mathcal{R}_D}$, where $P^*(\theta)$ is an optimal solution of the inner maximization problem defining $h(\theta)$.*

Proof. Let $\theta' \geq 0$ and let $P^*(\theta')$ be an optimal solution of the inner maximization problem defining $h(\theta') : h(\theta') = \langle Q, P^*(\theta') \rangle - \sum_{r \in \mathcal{R}_D} \theta'_r g_r(P^*(\theta'))$. Let a given θ . Since $P^*(\theta)$

is a feasible solution of the inner maximization problem defining $h(\theta)$:

$$h(\theta) \geq \langle Q, P^*(\theta) \rangle - \sum_{r \in \mathcal{R}_D} \theta_r g_r(P^*(\theta))$$

Substituting $\langle Q, P^*(\theta) \rangle$ by $h(\theta) + \sum_{r \in \mathcal{R}_D} \theta_r g_r(P^*(\theta))$, we have :

$$\begin{aligned} h(\theta) &\geq h(\theta) + \sum_{r \in \mathcal{R}_D} \theta_r g_r(P^*(\theta)) - \sum_{r \in \mathcal{R}_D} \theta'_r g_r(P^*(\theta)) \\ &\geq h(\theta) + \sum_{r \in \mathcal{R}_D} (\theta_r - \theta'_r) g_r(P^*(\theta)) \end{aligned}$$

□

The conic bundle algorithm iterates and stores the sub-gradients of h at the iterates θ_k in a bundle, and uses them to construct a piecewise linear lower approximation $\hat{h}_{\mathcal{R}_D}$ that is optimized at each iteration to generate the next iterate.

Algorithm 2 Conic bundle method for solving $\mathbf{SDP}_{CB}$

Require: $Q, \mathcal{R}, (Q_r, d_r)$ for all $r \in \mathcal{R}, \mathcal{R}_D, \hat{\theta}, u$, stopping criteria C_1 , not sufficient criteria C_2 .

Ensure: Certified lower bound

- 1: Set $\theta_0 = \hat{\theta}, k = 0, S = \emptyset$.
 - 2: Compute $h(\theta_0), P^*(\theta_0)$ and $g_r(P^*(\theta_0))$ for all $r \in \mathcal{R}_D$.
 - 3: **while** continue **do**
 - 4: Update the bundle S with the new sub-gradient $g(P^*(\theta_k))$ and the corresponding function value $h(\theta_k)$.
 - 5: Minimize $\hat{h}_{\mathcal{R}_D, k}(\theta) + \frac{u}{2} \|\theta - \theta_k\|^2$ and define θ_{k+1} as the minimizer.
 - 6: Compute $h(\theta_{k+1}), P^*(\theta_{k+1})$ and $g_r(P^*(\theta_{k+1}))$ for all $r \in \mathcal{R}_D$.
 - 7: **if** $\|h(\theta_{k+1}) - h(\theta_k)\| \leq C_1$ **then**
 - 8: stop.
 - 9: **else if** $\|h(\theta_{k+1}) - h(\theta_k)\| > C_2$ **then**
 - 10: Set $\hat{\theta} = \theta_{k+1}$.
 - 11: **else**
 - 12: Set $\hat{\theta} = \theta_{k+1}$.
 - 13: **end if**
 - 14: Set $k = k + 1$.
 - 15: **end while**
 - 16: **return** $h(\hat{\theta}_k)$
-

Note that the additional $\frac{u}{2} \|\theta - \theta_k\|^2$ term in the optimization problem is a proximal term that ensures the stability of the algorithm and prevents it from making large jumps in the dual space.

5.4.2 Dynamic conic bundle method

Note that the number of constraints in $\mathcal{R}$ can be very large, especially considering the RLT constraints (see number of constraints comparison in section 4.4). Noticing that

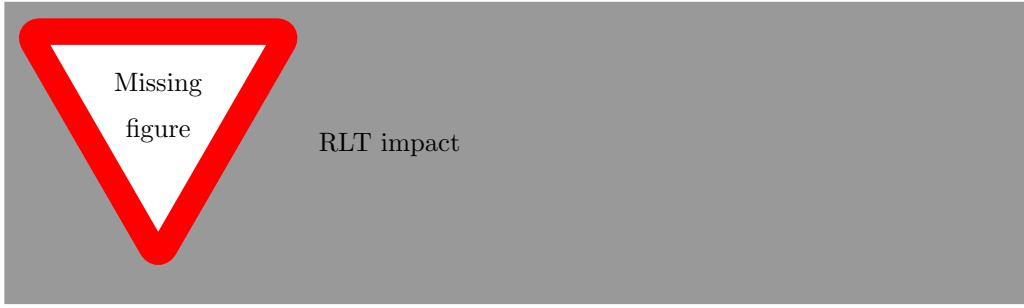


Figure 8: Caption

RLT constraints often drastically improve the bound when added sufficiently, but that the computing time of the SDP drastically increases with the number of constraints, we can consider to dualize all RLT constraints and to solve the resulting problem with a conic bundle method. The marginalised augmentation of the optimal value when augmenting the proportion p of RLT constraints is shown in Figure **A RAJOUTER** for a given network and a given input. We can see that, even if the first added RLT constraints have better impact on the optimal value, some RLT constraints ranked as least important by the heuristic, can still have a significant impact on the optimal value when added, and that the impact of the RLT constraints is not necessarily correlated with their ranking by the heuristic.

Thus the inner maximization problem defining $h(\theta)$ can be difficult to solve, when $\mathcal{R}_D$ is an important subset of $\mathcal{R}$. As the only interesting constraints in $\mathcal{R}_D$ are the ones that are active at the optimal solution, we can develop a dynamic conic bundle method that changes the subset $\mathcal{R}_D$ at each iteration by adding the most violated constraints and removing the least violated ones (with lagrangian multipliers λ_r near zero).

6 Conclusion

References

- Agler, J. et al. (1988). “Positive semidefinite matrices with a given sparsity pattern”. In: *Linear Algebra and its Applications* 107, pp. 101–149.
- Bai, Jia-Hau et al. (June 2025). *CAPM: Fast and Robust Verification on Maxpool-based CNN via Dual Network*. arXiv:2407.09550 [cs.CV]. DOI: [10.48550/arXiv.2407.09550](https://doi.org/10.48550/arXiv.2407.09550). URL: <http://arxiv.org/abs/2407.09550> (visited on 06/22/2026).
- Batten, Ben et al. (Aug. 2021). “Efficient Neural Network Verification via Layer-based Semidefinite Relaxations and Linear Cuts”. en. In: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. Montreal, Canada: International Joint Conferences on Artificial Intelligence Organization, pp. 2184–2190. ISBN: 978-0-9992411-9-6. (Visited on 04/23/2025).
- Bunel, Rudy et al. (2020). “Branch and Bound for Piecewise Linear Neural Network Verification”. en. In: *JMLR*.
- Cheng, Chih-Hong, Georg Nührenberg, and Harald Ruess (2017). “Maximum Resilience of Artificial Neural Networks”. In: *Automated Technology for Verification and Analysis*. Ed. by Deepak D’Souza and K. Narayan Kumar. Vol. 10482. Series Title: Lecture

- Notes in Computer Science. Cham: Springer International Publishing, pp. 251–268. ISBN: 978-3-319-68166-5 978-3-319-68167-2. (Visited on 06/03/2025).
- Chiu, Hong-Ming and Richard Y. Zhang (July 2023). “Tight Certification of Adversarially Trained Neural Networks via Nonconvex Low-Rank Semidefinite Relaxations”. en. In: *ICML*. PMLR, pp. 5631–5660. (Visited on 11/14/2025).
- Dathathri, Sumanth et al. (2018). “Enabling certification of verification-agnostic networks via memory-efficient semidefinite programming”. en. In: *NeurIPS*.
- Ferrari, Claudio et al. (Apr. 2022). *Complete Verification via Multi-Neuron Relaxation Guided Branch-and-Bound*. en. arXiv:2205.00263 [cs]. (Visited on 05/05/2025).
- Fischetti, Matteo and Jason Jo (July 2018). “Deep neural networks and mixed integer linear optimization”. en. In: *Constraints* 23.3, pp. 296–309. ISSN: 1383-7133, 1572-9354. (Visited on 06/12/2025).
- Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy (Mar. 2015). *Explaining and Harnessing Adversarial Examples*. en. arXiv:1412.6572 [stat]. (Visited on 05/10/2025).
- Gowal, Sven, Krishnamurthy Dvijotham, Robert Stanforth, Rudy Bunel, et al. (Aug. 2019). *On the Effectiveness of Interval Bound Propagation for Training Verifiably Robust Models*. en. (Visited on 05/26/2025).
- Gowal, Sven, Krishnamurthy Dvijotham, Robert Stanforth, Timothy Mann, et al. (July 2019). “A Dual Approach to Verify and Train Deep Networks”. In: *IJCAI*. IJCAI, pp. 6156–6160.
- Henriksen, Patrick and Alessio Lomuscio (Aug. 2021). “DEEPSPLIT: An Efficient Splitting Method for Neural Network Verification via Indirect Effect Analysis”. en. In: *IJCAI*. Montreal, Canada: IJCAI, pp. 2549–2555. ISBN: 978-0-9992411-9-6. (Visited on 05/14/2025).
- Huang, Xiaowei et al. (2017). “Safety verification of deep neural networks”. In: *International conference on computer aided verification*. Springer, pp. 3–29.
- Jaekle, Florian, Jingyue Lu, and M. Pawan Kumar (July 2021). *Neural Network Branch-and-Bound for Neural Network Verification*. en. arXiv:2107.12855 [cs]. (Visited on 05/05/2025).
- Katz, Guy, Clark Barrett, et al. (May 2017). “Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks”. en. In: *Computer Aided Verification*. arXiv:1702.01135 [cs]. (Visited on 04/28/2025).
- Katz, Guy, Derek A. Huang, et al. (2019). “The Marabou Framework for Verification and Analysis of Deep Neural Networks”. en. In: *Computer Aided Verification*. Ed. by Isil Dillig and Serdar Tasiran. Vol. 11561. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, pp. 443–452. ISBN: 978-3-030-25539-8 978-3-030-25540-4. (Visited on 04/28/2025).
- Lan, Jianglin, Yang Zheng, and Alessio Lomuscio (June 2022). “Tight Neural Network Verification via Semidefinite Relaxations and Linear Reformulations”. en. In: *AAAI* 36.7, pp. 7272–7280. ISSN: 2374-3468, 2159-5399. (Visited on 04/28/2025).
- (June 2023). “Iteratively Enhanced Semidefinite Relaxations for Efficient Neural Network Verification”. en. In: *AAAI* 37.12, pp. 14937–14945. ISSN: 2374-3468, 2159-5399. (Visited on 11/25/2025).
- Lu, Jingyue and M. Pawan Kumar (Dec. 2019). “Neural Network Branching for Neural Network Verification”. en. In: *ICLR*. (Visited on 05/14/2025).
- McCormick, G.P. (1976). “Computability of global solutions to factorable non-convex programs: Part I - Convex underestimating problems”. In: *Mathematical Programming* 10.1, pp. 147–175.
- Mirman, Matthew, Timon Gehr, and Martin Vechev (Oct. 2018). “Differentiable Abstract Interpretation for Provably Robust Neural Networks”. In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas

- Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, pp. 3578–3586. URL: <https://proceedings.mlr.press/v80/mirman18b.html>.
- Raghunathan, Aditi, Jacob Steinhardt, and Percy Liang (2018). “Semidefinite relaxations for certifying robustness to adversarial examples”. en. In: *NeurIPS*.
- Salman, Hadi et al. (n.d.). “A Convex Relaxation Barrier to Tight Robustness Verification of Neural Networks”. en. In: *NeurIPS* ().
- Singh, Gagandeep et al. (Jan. 2019). “An abstract domain for certifying neural networks”. en. In: *Proceedings of the ACM on Programming Languages* 3.POPL, pp. 1–30. ISSN: 2475-1421. (Visited on 06/04/2025).
- Tjeng, Vincent, Kai Xiao, and Russ Tedrake (Feb. 2019). “Evaluating Robustness of Neural Networks with Mixed Integer Programming”. In: *ICLR*. (Visited on 05/30/2025).
- Vandenbergh, Lieven and Martin S. Andersen (May 2015). “Chordal Graphs and Semidefinite Optimization”. en. In: *Foundations and Trends in Optimization* 1.4, pp. 241–433. ISSN: 2167-3888, 2167-3918. (Visited on 03/12/2026).
- Wang, Shiqi et al. (2021). “Beta-CROWN: Efficient Bound Propagation with Per-neuron Split Constraints for Neural Network Robustness Verification”. en. In: *NeurIPS*.
- Weng, Tsui-Wei et al. (2018). “Towards Fast Computation of Certified Robustness for ReLU Networks”. en. In: *ICML*.
- Wong, Eric and J Zico Kolter (2018). “Provable Defenses against Adversarial Examples via the Convex Outer Adversarial Polytope”. en. In: *ICML*.
- Wu, Haoze et al. (2024). “Marabou 2.0: A Versatile Formal Analyzer of Neural Networks”. en. In: *Computer Aided Verification*. Ed. by Arie Gurfinkel and Vijay Ganesh. Vol. 14682. Series Title: Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, pp. 249–264. ISBN: 978-3-031-65629-3 978-3-031-65630-9. (Visited on 05/26/2025).
- Xu, Kaidi, Zhouxing Shi, et al. (2020). “Automatic Perturbation Analysis for Scalable Certified Robustness and Beyond”. en. In: *NeurIPS*.
- Xu, Kaidi, Huan Zhang, et al. (Mar. 2021). “Fast and Complete: Enabling Complete Neural Network Verification with Rapid and Massively Parallel Incomplete Verifiers”. en. In: *ICLR*. (Visited on 05/26/2025).
- Zhang, Huan, Shiqi Wang, et al. (2022). “General Cutting Planes for Bound-Propagation-Based Neural Network Verification”. en. In: *NeurIPS*.
- Zhang, Huan, Tsui-Wei Weng, et al. (2018). “Efficient Neural Network Robustness Certification with General Activation Functions”. en. In: *NeurIPS*.
- Zhang, Richard Y (2020). “On the Tightness of Semidefinite Relaxations for Certifying Robustness to Adversarial Examples”. en. In: *NeurIPS*. NeurIPS.